Automating Software Supply Chain Security: AI-Powered Threat Detection in DevSecOps

Marlon Brenes Rojas
Student ID: 1314316
Master's in Cybersecurity
NYIT - Vancouver
mbrenesr@nyit.edu

***Abstract*—This project aims to design and implement an automated software supply chain security solution leveraging AI-driven threat detection in DevSecOps. The focus is predeployment alerts, ensuring real-time monitoring and threat detection within the CI/CD pipeline to improve security before applications reach production.**

I. Introduction

The speed of product development in business models demands that security be built into the software supply chain from the start of projects. Elaborate attacks such as SolarWinds and Log4j have alerted us to the need for secure and automated processes. Current tools help us to identify threats and vulnerabilities, and frameworks like [1]OWASP help us address them, but they often lack seamless integration into the action of typical DevOps teams. This project proposes a cohesive and automated framework that integrates these tools and methodologies, adding artificial intelligence-based detection tools and building on existing work and standards, such as [5] NIST SP 800-204D and the [1] OWASP DevSecOps framework. This study follows the journey from identifying gaps in supply chain security to proposing a practical AI-powered solution and testing it within a simulated DevSecOps environment.

II. Problem Statement

The objective is to establish automated flow tests that review and propose necessary modifications to enhance product security and reduce vulnerabilities by incorporating security measures into a DevSecOps pipeline. This framework will deliberately showcase vulnerabilities and weaknesses inherent in existing solutions through the integration of various tools. These tools include:

GitHub Actions: A CI/CD (Continuous Integration/Continuous Delivery) platform within the GitHub ecosystem, enabling the automation of software development workflows such as building, testing, and deploying code [16].

Microsoft Copilot Security: An AI-powered chatbot acting as a digital assistant to aid with diverse tasks across multiple platforms [8].

Snyk AI: The application of Artificial Intelligence and Machine Learning by Snyk to improve its developer security platform. Snyk's AI-driven tools assist in identifying, prioritizing, and automatically remediating vulnerabilities in code, open-source dependencies, containers, and Infrastructure as Code [10].

Detect-secrets: A tool designed to scan code repositories for sensitive information like API keys, passwords, and other secrets that could be unintentionally included in code commits [17].

CodeQL: A language and toolchain utilized for comprehensive code analysis [13].

Vulnerability Scanning: Processes for identifying security weaknesses in systems and applications [13].

Static Application Security Testing (SAST): A testing methodology that analyzes source code to identify potential security vulnerabilities before runtime [15].

ZAP Proxy (OWASP ZAP): A Dynamic Application Security Testing (DAST) tool used to find vulnerabilities in running web applications by intercepting and inspecting HTTP/HTTPS traffic [15].

KubeSec: A tool focused on the security analysis of Kubernetes configurations, identifying potential misconfigurations [18].

Interactive Application Security Testing (IAST): A security testing approach that combines elements of SAST and DAST by monitoring application behavior during runtime to identify vulnerabilities [18].

Trivy: A scanner specifically designed for Docker infrastructure vulnerability scanning [14].

OWASP Antipatterns: Known problematic design or implementation patterns in software development that can lead to security vulnerabilities.

Secure Deployment Mechanisms: Practices and tools aimed at deploying applications in a secure and reliable manner, utilizing technologies such as ArgoCD [19] and Kubernetes [20].

III. Review of Related Work

The DevSecOps methodology emphasizes the need for continuous security testing. Previous studies [7] Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines, have shown that the methodology improves vulnerability detection and helps to have a more efficient approach. Over the years, organizations have invested significant resources in monitoring and securing the software development life cycle (SDLC). In an increasingly

volatile market with greater demand for quality products, the evolution of DevSecOps from traditional DevOps has increased. DevSecOps integrates security teams into the early phases of the SDLC, ensuring that security is a central aspect rather than revisiting this need in the final stages of the project.

TABLE I
RECOMMENDED CONTINUOUS SECURITY TESTING ACTIVITIES IN EACH PHASE

Phase	Security Activities
Plan	Test planning
Code	Test development Static Application Security Testing (SAST)
Build	Software Composition Analysis (SCA) Static Application Security Testing (SAST)
Test	Dynamic Application Security Testing (DAST) Interactive Application Security Testing (IAST) Vulnerability scanning
Release	Vulnerability scanning Penetration testing
Deploy	Vulnerability scanning Penetration testing
Operate	Red & Blue team exercises
Monitor	Vulnerability scanning

According to [3] Automation and DevSecOps: Streamlining Security Measures in Financial System, the approach of DevSecOps is one of the most widely used frameworks in the financial sector, where security is embedded throughout the SDLC to ensure compliance and risk mitigation. This contrasts with traditional software development models, where security is often addressed as a final step. The authors emphasize proactive and continuous security measures to reduce manual testing and improve automated resilience through SAST and DAST techniques.

As shown in Fig. 1, the DevSecOps workflow consists of key security stages integrated within the CI/CD pipeline.

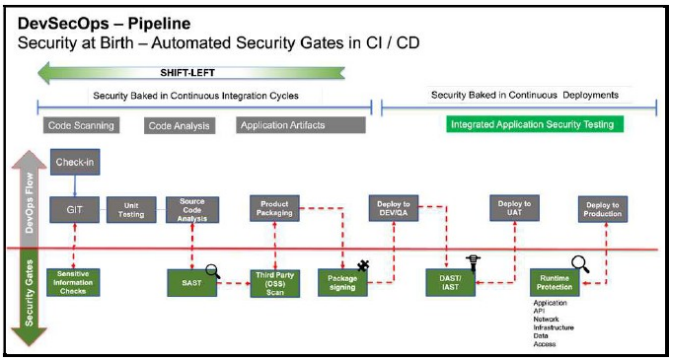


Fig. 1. DevSecOps Workflow in CI/CD Pipeline, adapted from [3].

Many of the tools used in previous research, such as GitHub, the OWASP CI/CD Framework, and various SAST/DAST tools, will also be implemented in our project's CI/CD pipeline.

As discussed in [9] AI for DevSecOps: A Landscape and Future Opportunities, AI-driven security techniques are evolving rapidly within DevSecOps. Despite the availability of many tools, coordinating and configuring them effectively remains a challenge. Strategy planning is crucial, as it can help integrate teams into the process. Using agents or artificial intelligence, such as that created by Microsoft in its Copilot product, could improve code quality right from code creation. Being able to identify potential vulnerabilities in early versions helps to address security issues proactively.

AI-powered security tools, such as Snyk [10], which focus on developer-first security, identify patterns and recommend best practices by analyzing historical code reviews and previously mitigated vulnerabilities. These tools assist with:

- Configuration and validation
- Code analysis
- Dependency management and security
- Security of Infrastructure as Code (IaC) deployments

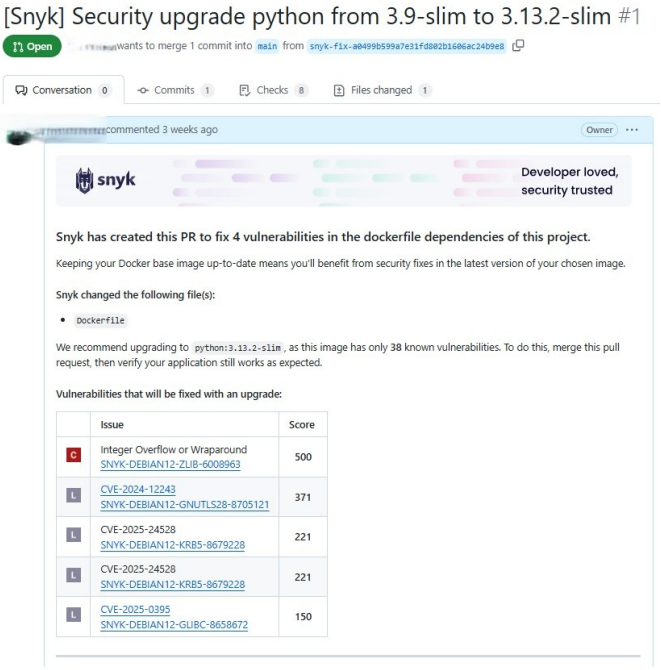


Fig. 2. Snyk's Automated Security Scanning, as shown in [10].

However, challenges remain. AI-driven threat detection may misidentify a good code as a threat or fail to suggest a fix if a pattern is missing from its already tested dataset. In addition, ethical issues arise as AI-driven automation grows; the strategy of reducing reliance on human expertise could be dangerous in the long run, with possible repercussions in breaking the compatibility of code underlying the primary code (other applications) and in the compression of self-generated code being complicated for a human to interpret, plus the labor market problem in supporting the future developed code. Additional challenges involve technological dependency and the interrelation of tools, while factors such as licensing fees, storage and logging requirements, the need for new

procedures, and the demand for specialized personnel can impact organizational costs and reliability.

As discussed in [4] Enhancing DevSecOps Pipelines with AI-Driven Threat Detection and Response, DevSecOps pipelines can further assist with AI-based threat detection and response mechanisms. Machine learning models significantly improve detection accuracy and automate security alerts, strengthening cyber resilience.

DevSecOps Pipelines						
Pre-Development	**Development**	**CI/CD**	**Deployment & Runtime**	**Continuous Learning and Improvement**	**Reporting and Visualization**	**Collaboration & Knowledge Sharing**
Threat Model Definition	Static Code Analysis	Automated Threat Detection	Dynamic Analysis	Feedback Loop	Dashboard and Alerts	Cross-Functional Collaboration
	Dependency Scanning		Real-Time Response			
Key Components						
AI Algorithms	Utilize various AI techniques, such as machine learning, deep learning, and natural language processing, to power threat detection, anomaly detection, and predictive analysis.					
Threat Intelligence Integration	Incorporate threat intelligence feeds and databases to enhance the accuracy of threat detection algorithms.					
Automated Orchestration	Develop automation scripts and tools that orchestrate responses to detected threats, including executing predefined actions and generating incident reports.					
Scalability and Performance	Design the framework to handle the scalability demands of large-scale applications, ensuring minimal impact on performance during real-time threat detection and response.					
Ethical Considerations	Ensure the AI-driven framework adheres to ethical guidelines and respects user privacy while performing tasks like code analysis and runtime monitoring.					

Fig. 3. AI-Driven Security Framework, as shown in [4].

In summary, while existing works demonstrate the value of integrating security into CI/CD pipelines, this project extends those efforts by incorporating AI-driven detection and focusing on automation, reproducibility, and measurable improvement in threat detection rates. Using mature models like Microsoft Flaml [22], the ability to correlate events in real-time and enhance actions based on those events surpasses human capacity, leading these technologies to become more integrated into our lives.

IV. Project Objectives

- Develop a CI/CD pipeline using GitHub Actions for automate cybersecurity test.
- Integrate automated security tools for scanning and alerting.
- Implement OWASP best practices for security scanning.
- Deploy applications using ArgoCD and Kubernetes.
- Simulate and mitigate common supply chain misconfiguration scenarios.
- Document results and challenges and recommend actions.

V. Proposed Solution

Considering the varying sizes of organizations, where the solution development staff can range from a few to thousands of people working and integrating their products continuously, organizations have seen the need to develop new strategies when it comes to giving quality value to their products, more specifically in the concept of Cybersecurity. Using technologies that are friendly to DevSecOps methodologies, such as Microsoft Github, security analysts can implement their knowledge through specialized tests within the integration flow such as Microsoft GitHub Actions, by creating a master pipeline where each phase is dependent on the previous one or from another strategy more focused on the principle of single responsibility, the cybersecurity professional can implement the following tools: Docker Hub test analysis, CodeQL, SnykCode, Trivy, detect secrets.

A. Implementation Process

Phase A: Source Code Management and Triggering

- **Step A1: Repository Initialization**
  - The project uses GitHub as the central repository with push and pull request triggers on the `main` branch.
  - Secrets are securely managed using `GitHub Secrets` to avoid hardcoded credentials.
- **Step A2: Pipeline Trigger Setup**
  - GitHub Actions are configured to automatically trigger builds on code updates.
  - Environmental variables such as `DOCKER_REGISTRY` and `IMAGE_NAME` are defined globally.

Phase B: Containerization and Build Process

- **Step B1: Source Code Checkout**
  - Uses `actions/checkout@v4` to clone the latest codebase from GitHub.
- **Step B2: Docker Build and Push**
  - Uses `docker/build-push-action@v4` to build and push images to Docker Hub.
  - Build arguments include secure runtime secrets like `APP_ENV`.
- **Step B3: Image Verification**
  - Pulls the image from the Docker registry to verify successful upload.

Phase C: Security Testing and Compliance

- **Step C1: Container Vulnerability Scanning**
  - Uses `Trivy` via `aquasecurity/trivy-action@0.28.0` to detect `CRITICAL` and `HIGH` vulnerabilities in the Docker image.
  - Results are converted to `SARIF` and uploaded to GitHub Security tab.
- **Step C2: Secrets Detection**
  - `detect-secrets` is used to scan for exposed credentials.
  - A baseline file is maintained to prevent regressions (`.secrets.baseline`).
- **Step C3: Static Code Analysis (SAST)**
  - `CodeQL` is initialized for Python-based static code analysis.
  - Security issues are uploaded and categorized in GitHub.
- **Step C4: AI-Based SAST and SCA with Snyk**
  - `Snyk CLI` runs both static analysis and open-source dependency scans.
  - SARIF reports from Snyk Code are uploaded to GitHub for visibility.
- **Step C5: Dynamic Application Security Testing (DAST)**

  - A Flask app is launched in the CI environment and scanned with `OWASP ZAP Full Scan`.
  - Reports are collected and uploaded as artifacts.
- **Step C6: AI Code Quality Scanning**
  - `CodeScanAI` evaluates code changes using LLM-based review tools and outputs actionable results.
- **Step C7: Kubernetes Configuration Security**
  - Kubernetes YAML manifests are scanned using `KubeSec` for misconfigurations and policy violations.
  - JSON output is converted to SARIF format for standard reporting.

Phase D: Continuous Deployment and Monitoring

- **Step D1: Kubernetes Deployment (ArgoCD)**
  - Although not explicitly shown in this pipeline, ArgoCD is integrated separately for GitOps-based deployment in the cluster.
- **Step D2: Incident Response and Observability**
  - Monitoring and alerting tools track container health and response status.
  - AI-generated auto-fix and auto-merge actions are triggered by secure pipelines (e.g., Snyk upgrade suggestions).

B. Challenges and Mitigation

- **Maintaining technological independence:** As discussed by the authors [6], Cyber Security Mesh Architecture (CSMA), discusses the challenges of modern, distributed systems and explores the benefits of decentralizing and modularizing security models. Key implementation principles are abstracting infrastructure, standardizing processes, and separating concerns. The author addresses moving from rigid security appliances toward a more adaptable and responsive cybersecurity framework. From this perspective, a dependency on technologies like those used in this project can be seen. Therefore, it is suggested that other technological actors be continually explored, as well as the single responsibility principle, in addition to using open standards like standard collecting logs compatible with the industry.
- **Scalability and Complexity:** Part of stable development is the growth of the software and, with it, the complexity of the same, given this nature, the complexity of maintaining a secure product over time. That is why well-defined policies are needed, in which all groups' efforts and limitations are apparent.
- **Keeping up with threats:** Maintaining a proper review of dependencies, infrastructure, and code is a daunting task from a daily operations perspective. Risk analyses are constantly necessary, either due to modifications of controls or changes of strategy at the time of the software architecture model.

C. Comparison with Alternatives

- Other solutions that could be mentioned are GitLab, SaltStak, Jenkins, Azure DevOps, or other open technologies community support. Many of them are old in the market; however, the maturity of each organization will dictate which is the best proposal; it is always recommended for human development where the technical, strategic, and business parts are developed, followed by the development of policies, standards, guidelines and procedures, and as a third pillar the implementation of the technology that best serves the organization.

D. Benefits and Outcomes

- Some of the benefits that could be mentioned are standardization of tests, alignment with legal regulations, cost reduction by not being manual, continuous analysis of vulnerabilities, and decentralization of security personnel to other activities.

E. Conclusion

- Taking into account the roadmap and the steps that will be developed, it could be indicated that in the continuous monitoring model, taking advantage of current technologies and new Artificial Intelligence models, the CI and CD business model is getting more efficient compared to manual tests where human error is always present.

VI. Methodology

The project methodology follows a structured, developed DevSecOps Framework, a phased approach designed to integrate security test measures at every stage of the software supply chain. The methodology focuses on the secure setup, deployment, and testing of CI/CD pipelines. The analysis and data collected and presented will be taken from the GitHub repository created for this project [11]

A. Pipeline Setup

In this phase, the goal is to establish a secure CI/CD pipeline that integrates automated security tools for continuous scanning and monitoring.

- Create a layered security analysis within the CI/CD pipeline using GitHub Actions, incorporating automated build and deployment checks.
- Integrate continuous security scanning tools like Trivy and CodeQL for real-time vulnerability detection and mitigation.

B. Security Enhancements

This phase focuses on implementing security best practices throughout the pipeline, ensuring security is integrated early and continuously.

- Implement best practices outlined in the OWASP DevSecOps Framework to ensure secure software delivery.
- Configure YAML-based declarative configurations for Kubernetes deployments using ArgoCD to streamline secure GitOps workflows.

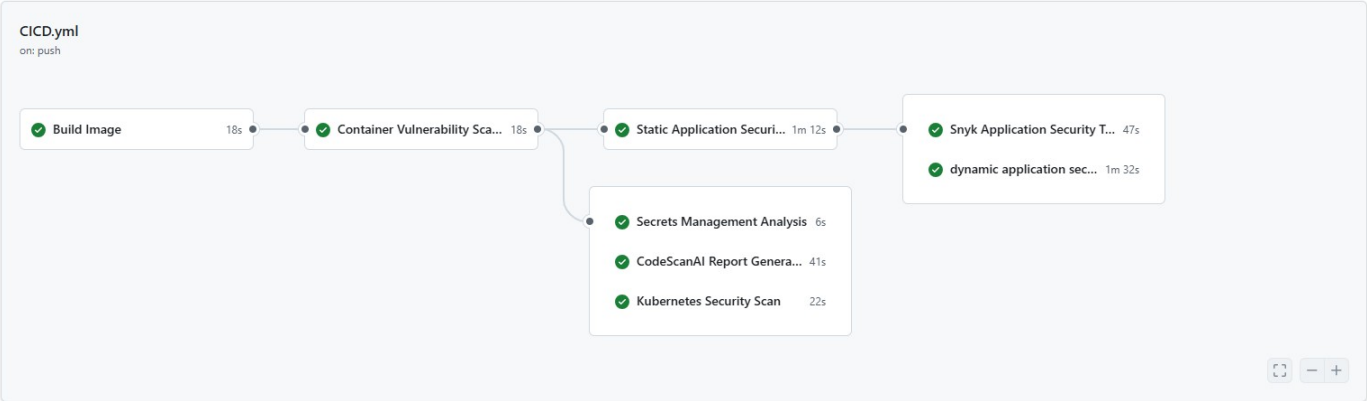


Fig. 4. Github Action Workflow, implemented as shown in [11].

C. Application Deployment

In this phase, the aim is to ensure that the application is securely deployed and monitored continuously, as exemplified in Table II.

- Use ArgoCD for GitOps-based continuous delivery, ensuring efficient and secure application deployment.
- Implement best practices for Kubernetes security to minimize attack surfaces and ensure secure configuration and network policies using the recommendations from the tool Kubesec [18].

D. Testing and Validation

The testing and validation phase identifies potential vulnerabilities within the software supply chain and evaluates the resilience of the CI/CD pipeline.

- Simulate supply chain misconfigurations in our intentionally vulnerable code and conduct penetration tests to assess the security posture of the pipeline using automation and AI methodologies in other suppliers such as Snyk.
- Analyze pipeline resilience by monitoring security alerts, logs, and system behavior during simulated attacks, allowing for effective response and mitigation as is show in figure 4.

Table II summarizes the security vulnerabilities identified during testing, categorized by severity level, along with the detection tools used.

The results from testing were analyzed by reviewing security alerts and logs, assessing the effectiveness of mitigation strategies, and evaluating the pipeline's behavior after the test phase action.

E. Test Procedure Design (Flowchart)

To further improve the system, the following additional measures need to be implemented:

- Create a procedure for initiating incident response on security event alerts, ensuring rapid containment of security breaches.
- Expand the scope of testing to simulate insider attacks and security vulnerabilities, covering a wider range of potential threats.

TABLE II
SUMMARY OF TOP 10 SECURITY VULNERABILITIES AND DETECTION TOOLS

Vulnerability	**Severity**	**Detection Tool**
zlib: Heap-based buffer overflow	Critical	Trivy
Uncontrolled command execution	Critical	CodeQL
Server-side template injection (SSTI)	Critical	CodeQL
CPAN: TLS verification missing	High	Trivy
Remote code execution in setuptools	High	Trivy
Regular expression DoS (ReDoS)	High	Trivy
Python-Werkzeug privilege escalation	High	Trivy
Flask: Permanent session cookie issue	High	Trivy
Reflected cross-site scripting (XSS)	Medium	CodeQL
Information exposure through exception messages	Medium	CodeQL
Top 10 Vulnerabilities Found	**Top 10 identified issues**	

- In case of using Agile methodologies, participate as soon as possible in the project dailies and iterations to express the findings from every code commit analyzed.

As shown in Fig. 5, the OWASP DevSecOps Guideline provides a structured approach to integrating security into DevOps workflows.

F. Data Analysis of Test Procedure Performed

This section presents the analysis of the CI/CD pipeline security and automated rollback performance. The tables below summarize key metrics obtained during testing. To implement the pipeline and run multiple tools and test cases, a project [11] was created in GitHub as a repository, and a pipeline was established using the methodology of Continuous Integration and Continuous Delivery suggested by OWASP [1] and the mature framework of OWASP SAMM [12].

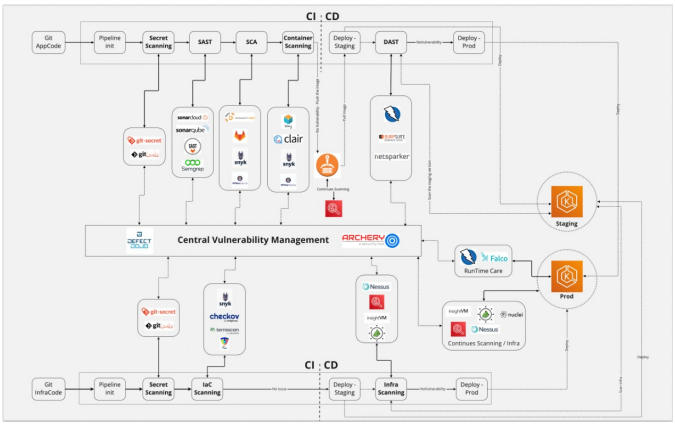


Fig. 5. OWASP DevSecOps Guideline, as shown in [1].

G. Methodology

This study evaluates the effectiveness of five security scanning tools within a DevSecOps CI/CD pipeline. The analysis focuses on detecting vulnerabilities injected into the test environment, including SQL injection (SQLi), hardcoded secrets, and dependency vulnerabilities. The tools tested are:

- **CodeQL** (Static Code Analysis)
- **Trivy** (Container and Dependency Scanning)
- **Snyk** (Open Source Security)
- **OWASP ZAP** (Analyze code in DAST technique)
- **Kubesec** (Analyze Infraestructure code like kubernetes file)

Each tool was tested against 50 runs of commit code, distributed across 10 test cases per tool, with known vulnerabilities used as ground truth in the intentional vulnerable code in our project. In Snyk's case, it's important to note the limitation on the number of runs per month. At the time the test case was performed, the free license limited usage to 100 runs per month.

H. Test Case Mapping

In the Table "Test Case Mapping to Detected Vulnerabilities" III, a Test ID column is shown, which indicates the number of the test case. The next column indicates which tool was used. The Detected Vulnerabilities column indicates what type of code vulnerability was identified in the scan phase of our CI/CD pipeline. The last column shows what was explicitly reported in the security tab of our GitHub repository.

I. Discussion and Limitations

- **CodeQL is a very useful and indispensable tool to have in the pipeline; however, its sensitivity should be complemented by other tools for more accurate results.**
- **Trivy, as mentioned in the reference [14], is a specialized tool for Docker technology. Depending on the nature of the project and its technologies, it could be practical to find different tools to enhance these technologies.**
- **I find Snyk AI to be the most accurate and helpful tool in our pipeline, and it should definitely be a standard part of our toolkit. However, to ensure technology independence, the suggestion is to maintain at least two tools that offer similar functionalities.**
- **OWASP ZAP is a crucial tool for the DAST approach, enabling the discovery of vulnerabilities through the inspection of HTTP/HTTPS traffic in a deployed environment.**
- **Kubesec is a useful tool; however, its reports offer suggestions and do not outline the required actions to enhance the scanned file.**

J. OWASP SAMM Alignment

This analysis aligns with the [12]**OWASP SAMM framework**, contributing to the following maturity levels:

- **Implementation:** Automated security scanning as part of CI/CD.
- **Governance:** Metrics-driven assessment for continuous improvement.
- **Verification:** Establishing a security feedback loop.

The findings provide insights into tool effectiveness within a secure SDLC, guiding organizations in DevSecOps adoption.

K. Discussion of Project and Results

1) Interpretation of Results:

- Analysis of security tool effectiveness and performance.
- Identification of trends in vulnerability detection and resolution.
- Comparison of observed results with expected outcomes.

2) Future Development and Improvements:

- Enhancing integration of security tools to reduce false positives.
- Improving automated rollback mechanisms based on risk assessment.
- Expanding test coverage to include additional vulnerability classes.

3) Challenges Faced:

- Integration difficulties with AI-powered security tools.
- Handling false positives and negatives in security scans.
- Managing pipeline performance while maintaining security.

4) Future Development and Improvement:

- **Enhancing AI-Driven Threat Detection and Response:** Improve AI models to better identify, classify, and mitigate security threats in real-time, reducing reliance on manual intervention.
- **Reducing False Positives through Machine Learning Improvements:** Implement adaptive learning mechanisms to refine threat detection accuracy, minimizing false positives and false negatives.
- **Expanding Security Monitoring Beyond the CI/CD Pipeline:** Extend security automation to runtime environments, container orchestration, and infrastructure as code (IaC) to ensure continuous protection.

TABLE III
TEST CASE MAPPING TO DETECTED VULNERABILITIES

Test ID	Tool	Tested Item	Detected Vulnerabilities	Explicitly Reported
TC-01	CodeQL	SQL Injection Detection	Server-Side Template Injection (SSTI)	Yes
TC-02	Detect-Secrets	Hardcoded Secrets	PASSWORD: "PuraVida!"	Yes
TC-03	Snyk	Dependency Vulnerabilities	Werkzeug RCE (CVE-2024-34069)	Yes
TC-04	OWASP ZAP	DAST Vulnerabilities	XSS, CSP Issues	Yes
TC-05	Trivy	Container Security	zlib Buffer Overflow	Yes
TC-06	Kubesec	IaC Security	K8s Misconfigurations	Partial
TC-07	Manual Audit	Security Headers	Missing CSP Headers	Yes
TC-08	CodeQL/Snyk	Authentication Controls	Weak Password Hashing	Yes
TC-09	CodeQL	Input Validation	Path Traversal	Yes
TC-10	Snyk	Outdated Libraries	Request v2.88.0	Yes
TC-11	Pre-commit Hooks	Code Quality	N/A	Not Reported
TC-12	Build System	Build Integrity	Dependency Chain Risks	Indirect
TC-13	Manual Test	Authorization Controls	Privilege Escalation Risks	No

VII. RESOURCES

A. Tools

The following tools were integrated into the CI/CD pipeline:

- GitHub Actions [16]
- Docker Hub [21]
- Snyk CLI [10]
- CodeQL [13]
- Trivy [14]
- Detect-Secrets [17]
- KubeSec [18]
- Kubernetes [20]
- ArgoCD [19]
- OWASP ZAP [15]

B. Software

The project used Docker containers, ArgoCD and Kubernetes (Minikube) to simulate deployment environments, along with secure image repositories.

C. Hardware

Experiments were conducted on local virtualized testbeds, including local virtual machines.

D. Data

Test cases were conducted using a combination of intentionally vulnerable Flask applications and publicly available open-source vulnerable code repositories.

VIII. PROJECT SCHEDULE

- **Week 1-2**: Research and define project scope; set up GitHub Actions.
- **Week 3-5**: Integrate Snyk and CodeQL; document initial findings.
- **Week 6-8**: Implement OWASP security practices; configure ArgoCD and Kubernetes.
- **Week 9-11**: Simulate supply chain attacks; refine security measures.
- **Week 12-14**: Document results; finalize project and presentation.

IX. CONTRIBUTION TO KNOWLEDGE

This project aims to:

- Demonstrate an automated secure DevSecOps pipeline.
- Highlight the integration of AI-driven security tools.
- Address gaps in software supply chain security.
- Provide recommendations for secure software delivery processes.

X. CONCLUSION

In conclusion, as stated in *NIST SP 800-204D* [5] Security and Privacy Controls for Information Systems and Organizations, Securing the software supply chain from a DevSecOps perspective requires well-defined strategies. Organizations must align their business and technology efforts to improve security, as key factors such as quality, cost, and speed of creating secure software are directly affected by these perspectives.

Continuous monitoring of the entire software supply chain is essential to maintaining a sound business strategy. This research has identified key challenges and proposed effective solutions for implementing an initial DevSecOps framework that incorporates security across all layers of the software supply chain.

Furthermore, the tools and methodologies discussed in this study adhere to the suggestions of the framework from OWASP [1]. While emphasizing automation to enhance security and efficiency, the general guidance implies that these automated tools still require supervision. By integrating AI-driven threat detection and continuous security testing, organizations can strengthen their DevSecOps pipelines, mitigate risks, and proactively address emerging security threats.

REFERENCES

[1] OWASP, *OWASP DevSecOps Guideline*, [Online]. Available: https://owasp.org/www-project-devsecops-guideline/.

[2] C. Feio, N. Santos, N. Escravana, and B. Pacheco, *An Empirical Study of DevSecOps Focused on Continuous Security Testing*, International Journal of DevSecOps Research, 2024.

[3] M. K. Kushwaha, P. David, and G. Suseela, *Automation and DevSecOps: Streamlining Security Measures in Financial System*, Department of Networking and Communications, SRM Institute of Science and Technology, Chennai, India, 2024. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10502917. [Accessed: Feb. 28, 2025].

[4] S. Yulianto and G. N. C. Ngo, *Enhancing DevSecOps Pipelines with AI-Driven Threat Detection and Response*, Doctor in Information Technology, School of Graduate Studies, AMA University, Quezon City 1106, Philippines, 2024. [Online]. Available: semi.yulianto2009@gmail.com, gncngo@amaes.edu.ph.
[5] National Institute of Standards and Technology (NIST), *NIST SP 800-204D: Security and Privacy Controls for Information Systems and Organizations*, NIST, 2020. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204D.pdf. [Accessed: Feb. 28, 2025].
[6] *Implementing CSMA – the Cloud Mindset*, SecurityInfoWatch.com, Endeavor Business Media LLC, May 2024. [Online]. Available: https://www.securityinfowatch.com/cybersecurity/article/55040906/implementing-csma-the-cloud-mindset.
[7] R. Chandramouli, F. Kautz, and S. Torres-Arias, *Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*, NIST Special Publication 800-204D, National Institute of Standards and Technology (NIST), 2024. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-204D.
[8] Microsoft Corporation, *Prompt crafting in Visual Studio Code*, Visual Studio Code Documentation, 2024. [Online]. Available: https://code.visualstudio.com/docs/copilot/prompt-crafting. [Accessed: Mar. 8, 2025].
[9] M. Fu, J. Pasuksmit, and C. Tantithamthavorn, *AI for DevSecOps: A Landscape and Future Opportunities*, ACM, vol. 1, no. 1, Art. , Sep. 2024, pp. 1-58. [Online]. Available: https://doi.org/10.1145/3712190. [Accessed: Mar. 8, 2025].
[10] Snyk Ltd., "Snyk: Developer-first security," Snyk Documentation, 2024. [Online]. Available: https://snyk.io. [Accessed: Mar. 8, 2025].
[11] Marlon Brenes, "DevSecOps Project Repository." [Online]. Available: https://github.com/BrenesRM/DevSecOps-NYIT-VAN
[12] OWASP, *OWASP Software Assurance Maturity Model (SAMM)*, [Online]. Available: https://owaspsamm.org/. Accessed: [Insert Date Accessed].
[13] GitHub, *CodeQL*, [Online]. Available: https://codeql.github.com/.
[14] Aqua Security, *Trivy*, [Online]. Available: https://github.com/aquasecurity/trivy.
[15] OWASP, *OWASP ZAP*, [Online]. Available: https://github.com/zaproxy/zaproxy.
[16] GitHub, *GitHub: Where the world builds software*, [Online]. Available: https://github.com/.
[17] PyPI, *Detect Secrets*, [Online]. Available: https://pypi.org/project/detect-secrets/.
[18] ControlPlane, *Kubesec*, [Online]. Available: https://github.com/controlplaneio/kubesec.
[19] Argo Project, *Argo CD Documentation*, [Online]. Available: https://argo-cd.readthedocs.io/en/stable/.
[20] Kubernetes, *Kubernetes Official Website*, [Online]. Available: https://kubernetes.io/.
[21] Kubernetes, *Docker Hub Official Website*, [Online]. Available: https://hub.docker.com/.
[22] Kubernetes, *Microsoft Flaml Official Website*, [Online]. Available: https://microsoft.github.io/FLAML/.

XI. Appendices

A. Supplemental Materials

- **Data Sheets:** Specifications of the security tools utilized in this project are as follows:
  - **CodeQL** v2.13.5 – Used for static code analysis to detect syntax and logic vulnerabilities in Python code.
  - **Trivy** v0.45.1 – Employed for container and dependency scanning, focusing on identifying known CVEs in Docker images.
  - **Snyk CLI** v1.1262.0 – Used for both SAST and Software Composition Analysis (SCA) of dependencies.
  - **Detect-Secrets** v1.4.0 – Integrated to identify and baseline exposed secrets in code commits.
  - **OWASP ZAP** Full Scan Action v0.12.0 – Utilized for dynamic application security testing (DAST) on a running Flask application.
  - **KubeSec Action** (latest from controlplaneio) – Applied for infrastructure-as-code (IaC) scanning of Kubernetes manifests.
  - **CodeScanAI Action** v1.0.1 – Used to generate LLM-based code quality and vulnerability reports based on OpenAI GPT models.

B. Step-by-Step Project Configuration Guide

1) **Environment Setup**

- Create a GitHub repository with the following structure:

```
/src
/tests
Dockerfile
requirements.txt
CICD.yaml
```

- Configure GitHub Secrets (Settings → Secrets → Actions):
  - `DOCKER_USERNAME`, `DOCKER_PASSWORD`
  - `SNYK_TOKEN`, `GITHUB_TOKEN`

2) **CI/CD Pipeline Configuration**

- Implement GitHub Actions workflow:

```
name: PIPELINE-CI
on: [push, pull_request]
jobs:
  build: # Docker image build
  dockertest: # Trivy container scan
  sast_analysis: # CodeQL SAST
```

- Enable required permissions in the workflow:

```
permissions:
  security-events: write
  actions: read
```

3) **Security Tool Integration**

- **CodeQL SAST Setup:**

```
- uses: github/codeql-action/init@v3
  with: languages: 'python'
```

- **Trivy Container Scanning:**

```
- uses: aquasecurity/trivy-action@0.2
  with: severity: 'CRITICAL,HIGH'
```

- **Snyk Dependency Scanning:**

```
- run: npm install -g snyk
- run: snyk monitor --all-projects
```

4) **Docker Deployment**

- Build secure Docker image:

```
FROM python:3.13.2-slim
RUN pip install --no-cache-dir -r
EXPOSE 5000
CMD ["python", "app.py"]
```

- Push to Docker Hub:

```
docker/build-push-action@v4 with:
```

5) **Validation & Testing**
   - Run full pipeline test:

```
git commit -am "Test trigger" &&
```

   - Verify results in GitHub:
     - Security tab: CodeQL/Trivy findings
     - Actions console: Pipeline execution logs
     - Artifacts: ZAP reports

Troubleshooting Tips:

- **Secret Detection Failures:** Update baseline after intentional secrets:

```
detect-secrets scan --update .secrets.baseline
```

- **Container Scan Errors:** Check base image vulnerabilities:

```
trivy image python:3.13.2-slim
```

- **SAST False Positives:** Customize CodeQL queries:

```
uses: github/codeql-action/analyze@v3
with: queries: security-and-quality
```

Alerts Report: Results of the Alerts collected:

Test Cases:

C. Explanation of Vulnerabilities

- **SQL Injection Prevention:** The test ensures that the application blocks attempts to inject SQL queries into endpoints.
- **XSS Mitigation:** This test checks that the application sanitizes user input to prevent XSS attacks.
- **Authentication Success/Failure:** Ensures that the login mechanism works correctly and that invalid credentials are handled properly.
- **Open Redirect Prevention:** Verifies that the application prevents users from being redirected to external untrusted sites.
- **Rate Limiting:** Ensures that the application limits the number of requests from the same source to prevent abuse.
- **File Read Traversal Prevention:** Confirms that the application prevents directory traversal attacks.
- **Deserialization Safety:** Ensures that the application properly handles deserialization, rejecting invalid or malicious payloads.

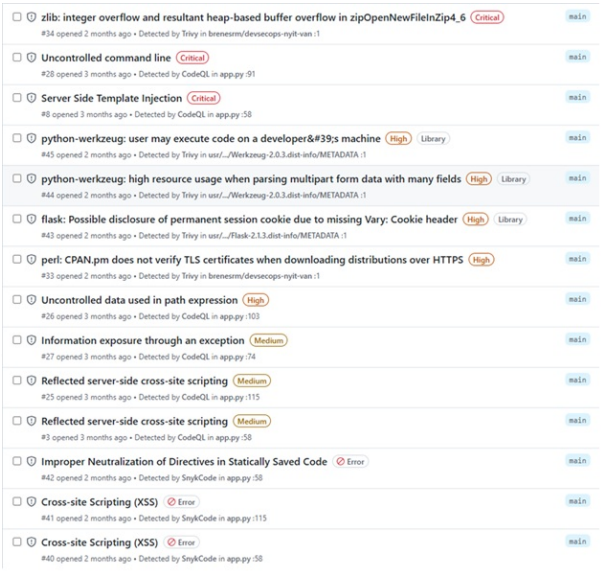


Fig. 6. Security Alerts Tab in GitHub

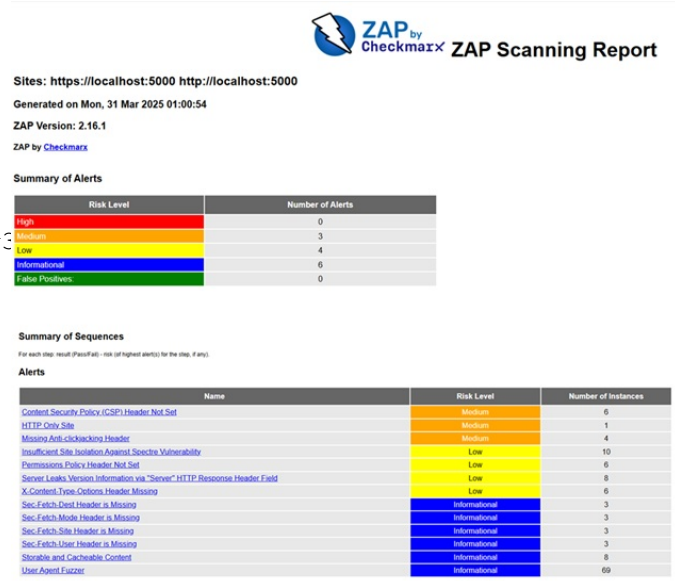


Fig. 7. ZAP Scanning Report

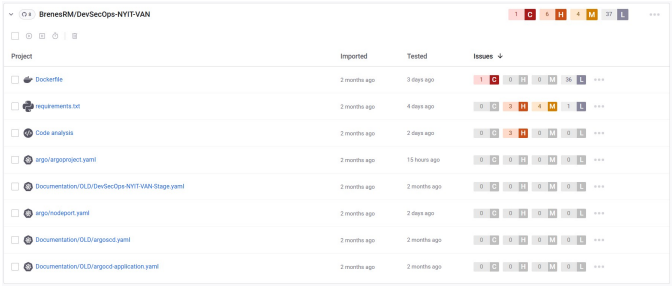


Fig. 8. Snyk Report

- **FP and FN:** False Positives (FP) and False Negatives (FN), impacting vulnerability detection accuracy. This section documents two cases observed in our DevSecOps pipeline testing:
  - **False Positive Example:** Tool: CodeQL, Tested Secure SQL Query.

    Detected Vulnerabilities: SQL Injection (False Positive)

    Explicitly Reported: Yes (False Positive)

    Scenario: The following code securely queries a database using parameterized queries. However, CodeQL incorrectly flags it as vulnerable to SQL Injection: [language=Python, caption=Secure SQL Query Misclassified as SQL Injection] @app.route("/user/¡username¿") def getuser(username): conn = sqlite3.connect(DATABASE) cursor = conn.cursor() query = "SELECT * FROM users WHERE username = ?" cursor.execute(query, (username,)) user = cursor.fetchone() conn.close() if user: return jsonify(id=user[0], username=user[1]) else: return jsonify(error="User not found"), 404
  - **False Negative Example:** Tool: Snyk, Detected Vulnerabilities: None (False Negative).

    Explicitly Reported: No

    Scenario: A Flask application uses an outdated Werkzeug package with a known Remote Code Execution (RCE) vulnerability (CVE-2024-34069). However, Snyk failed to detect it. Could be for the limitation of the free license. [language=Python, caption=Vulnerable Dependency in Requirement] Vulnerable dependency in requirements.txt Werkzeug==2.2.0

XII. Initial Proposal with Advisor Comments

A. Comments Initial Proposal

Great proposal, thank you! The only issue is the IEEE format, which is not exactly that. Also, following the structure of the proposal which can be found in the assignment. I need to check the scope with you once again and make sure the complexity and testing parts.

To focus next:

Making the report to the IEEE format mentioned Check the structure of each assignment and prepare based on that Explanation of the AI-powered threat detection Sara Khanchi , Feb 24 at 8:46am

B. Comments midd-tearm

Great job, I can see a lot of progress, and it's clear you've put a lot of effort into the report. The related works section is very well-researched, and you've explained each part clearly. You're on the right track! One suggestion for improvement would be to tie everything together more cohesively, as if you're telling a story throughout the paper. This will help the flow and make your argument even stronger.

Regarding the results section, I'd like to discuss it further with you since some parts were a bit unclear to me as well. Let's go over it together to make sure it's polished and ready for the final submission and publication.

A few things to keep in mind:

It would be helpful to add citations for each tool you mentioned in the problem statement. The related works section is well-written and touches on the challenges; just be sure to add a brief summary and conclusion at the end to tie it all together. In the implementation process, consider adding a visual representation of your process, and label each part (A, B, C, etc.) for clarity before explaining each one in detail. Have you thought about how to incorporate insider attacks into your work? That would be interesting. Don't forget to include the mandatory appendices you referenced in the report, and be sure to include a link to your GitHub repository. You're making fantastic progress, and with just a few tweaks, this will be ready. I'm excited to see how it all comes together!

Sara Khanchi , Mar 17 at 5:33pm For your presentation, it would be great to provide more details on the prerequisites, such as the tools you've used throughout the project. Additionally, consider incorporating some of your results into the slides to highlight your findings. You've put in a lot of effort, and showcasing that in your presentation will help convey the depth of your work more effectively. Keep up the great effort!

XIII. Progress Reports with Comments

A. Comments